

Heurísticas baseadas em Simulated Annealing e Algoritmo Genético para solução do Sudoku

Lademir Silva Rodrigues Júnior - 102018

Departamento de informática
INF 284 - Metaheurística
Universidade Federal de Viçosa (UFV) – Viçosa, MG – Brasil
lademir.junior@ufv.br

1. Introdução

O Sudoku é um popular quebra-cabeça lógico que desafia os jogadores a preencher uma grade 9x9 com números de 1 a 9, garantindo que cada coluna, cada linha e cada subgrade 3x3 contenham todos os números sem repetição. Surgiu pela primeira vez no Japão na década de 1980, mas se tornou um fenômeno global nos anos 2000. O sucesso do Sudoku pode ser atribuído à sua simplicidade e à capacidade de ser resolvido sem o uso de cálculos matemáticos complexos. Desde então, o Sudoku tem sido publicado em jornais, revistas e ganhou popularidade em aplicativos de dispositivos móveis, desafiando jogadores em todo o mundo a usar seu raciocínio lógico e habilidades de dedução para encontrar a solução correta.

Embora o Sudoku possa ser resolvido por meio de técnicas de busca exaustiva, como backtracking, dependendo da dificuldade do tabuleiro, esse método pode ser extremamente demorado e ineficiente. Para abordar esse problema, diversas técnicas baseadas em metaheurísticas já foram propostas, incluindo do SA(Simulated Annealing) e Algoritmo Genético.

Neste trabalho, exploraremos a eficiência e a aplicabilidade desses dois métodos de otimização na resolução do Sudoku. Analisaremos o desempenho de ambos os algoritmos em termos de tempo de execução e precisão na obtenção da solução correta.

2. Trabalhos Relacionados

A resolução do Sudoku é um assunto já bastante trabalhado na literatura há um tempo considerável. Na tabela 1 podemos observar alguns desses trabalhos e que foram usadas abordagens tanto de solução única (*s-metaheuristic*) quanto de abordagens populacionais (*p-metaheuristic*) além de uma mesclagem dessas duas técnicas, que seriam os métodos híbridos.

Tabela 1. Estudos relacionados com o tema

Número	Algoritmo	Tipo de Algoritmo	Referência
1	Variable Neighborhood Search (VNS)	Solução Única	Khorshid, 2014

2	Tabu Search (TB)		Simone & Romero, 2013
3	Simulated Annealing (SA)		Lewis, 2007 – Simone & Romero, 2013
4	Genetic Algorithm (GA)	Populacionais	Mantere & Koljonen, 2006
5			Mantere & Koljonen, 2007
6			Deng et al., 2011
7			Deng et al., 2012
8			Deng & Li, 2013
9			Deng et al., 2013
10			Geem, 2007
11	Harmony Search (HS)		Mandal & Sadhu, 2011
12			Mandal & Sadhu, 2013
13	Cultural Algorithm (CA)		Mantere & Koljonen, 2008a
14	Particle Swarm Optimization (PSO)		Hereford & Gerlach, 2008
15	Artificial Bee Colony (ABC)		Pacurib et al., 2009
16	Ant Colony (AC)		Asif & Baig, 2009
17	Differential Evolution (DE)		Boryczka & Juszczuk, 2012
18	Genetic Programming Approach (GPA)		Pillay, 2012
19	Cuckoo Search (CS)		Soto et al., 2014
20	Wisdom of Artificial Crowds		Hughes & Yampolskiy, 2012
21	Cultural Genetic Algorithm (CGA)	Híbridos	Perez & Marwala, 2008
22	Repulsive Particle Swarp Optimization (RPSO)		
23	Quantum Simulated Annealing (QSA)		Soto, 2013
24	Genetic Algorithm with Simulated Annealing(GASA)		

Os estudos anteriores que mais inspiraram esse trabalho foram os relacionados a Simulated Annealing, Algoritmo Genético e a versão híbrida dessas duas técnicas.

3. Metodologia proposta

Antes de mostrar como foi feita a abordagem das heurísticas baseadas nas citadas anteriormente, é importante falar sobre como seria feita uma solução inicial, como seria feita a função de avaliação de uma solução, e também como seria uma abordagem de vizinhança para uma dada solução.

3.1. Solução inicial

Figura 1. Tabuleiro com um quebra-cabeça no estado inicial

Fitness: 0

9			2		4			
3			5	8			9	7
8		5		3			6	
7	8		4		2	9		3
6			7	5		1		
4	5							
		8				5	2	
	4			7	5		3	
5		9	6		8			

A partir de uma instância no estado inicial, como podemos observar na figura 1, temos duas maneiras de criar uma solução inicial, ou seja, de preencher todo o tabuleiro de uma maneira para que seja usado nas técnicas de otimização, entretanto, para facilitar a resolução do problema, para qualquer método de obter uma solução inicial, iremos respeitar o critério de em cada subgrade não ter repetições, ou seja, nunca iremos colocar números duplicados nesses espaços.

Para facilitar a visualização das células iniciais do tabuleiro, vamos indicá-las pelo fundo azul.

3.1.1. Solução aleatória

Podemos partir para uma solução inicial aleatória, ou seja, para cada espaço dentro da subgrade, escolhe-se um número aleatório que atenda o critério de não ter esse número na subgrade, para que não ocorra repetição de números dentro de uma subgrade. Podemos observar um exemplo de uma solução obtida nesse modo na figura 2, em que é uma solução inicial para o tabuleiro mostrado na figura 1.

Figura 2. Tabuleiro com uma solução inicial aleatória

Fitness: 40

	9	7	2	2	9	4	2	5	3
	3	1	4	5	8	1	1	9	7
	8	6	5	6	3	7	8	6	4
	7	8	2	4	6	2	9	6	3
	6	3	9	7	5	1	1	8	2
	4	5	1	3	9	8	5	7	4
	7	2	8	4	3	2	5	2	9
	1	4	6	1	7	5	1	3	6
	5	3	9	6	9	8	7	8	4

3.1.2. Solução gulosa

Outra maneira de obter uma solução inicial seria uma abordagem gulosa, ou seja, analisa os passos possíveis e toma aquele que seria o melhor baseado em algum critério, nesse caso, o critério utilizado seria o custo em que o número adicionaria no custo total da solução, a forma como é calculado o custo será abordado na próxima seção – podemos observar o funcionamento dessa função na figura 3.

Figura 3. Pseudocódigo para solução inicial gulosa

Algorithm 1 Greedy_Solution

```

for cada grid em grids do
   $digits \leftarrow \{1, 2, \dots, n\}$  {conjunto de todos os dígitos possíveis}
  for cada célula em grid do
    if célula contém um valor fixo then
      continuar para a próxima célula
    end if
     $min\_el \leftarrow 0$  {inicializar o menor elemento}
     $min\_cel\_cost \leftarrow 100$  {inicializar o custo mínimo da célula}
    for cada dígito em digits do
       $d\_cost \leftarrow calcular\_custo(linha, coluna, dígito, board)$  {calcular o custo de colocar o dígito na célula}
      if  $d\_cost < min\_cel\_cost$  then
         $min\_el \leftarrow dígito$  {atualizar o menor elemento}
         $min\_cel\_cost \leftarrow d\_cost$  {atualizar o custo mínimo da célula}
      end if
    end for
    remover  $min\_el$  de digits {remover o elemento escolhido dos dígitos disponíveis, para que não tenha repetição de dígitos}
     $board[célula] \leftarrow min\_el$  definir o valor da célula como  $min\_el$ 
  end for
end for
return board

```

Um ponto importante da abordagem gulosa é que, por ter a característica determinística, sempre será obtido o mesmo resultado, portanto, podemos observar na figura 4 a solução inicial gulosa para a instância representada na figura 1.

Figura 4. Tabuleiro com solução gulosa

Fitness: 17

9	1	6	2	1	4	3	5	8	
3	2	4	5	8	6	1	9	7	
8	7	5	9	3	7	2	6	4	
7	8	1	4	6	2	9	4	3	
6	3	2	7	5	9	1	8	2	
4	5	9	1	3	8	6	7	5	
1	6	8	3	4	1	5	2	9	
2	4	3	9	7	5	8	3	1	
5	7	9	6	2	8	4	6	7	

3.2. Função de avaliação

Agora que observamos como uma solução inicial pode ser obtida, é importante entendermos como uma solução atual pode ter um custo associado, com isso, utilizaremos uma função de avaliação para obter um custo para uma dada solução, portanto, o objetivo das heurísticas é o de atingir 0 na função de avaliação.

Figura 5. Pseudocódigo da função de avaliação

Algorithm 2 fit

```

1:  $fit \leftarrow 0$ 
2:  $rows \leftarrow obter\_linhas()$  {Obter as linhas do Sudoku}
3:  $cols \leftarrow obter\_colunas()$  {Obter as colunas do Sudoku}
4:  $row\_count \leftarrow map()$ 
5:  $col\_count \leftarrow map()$ 
6: for cada linha em rows do
7:    $row\_count[linha] \leftarrow contar\_elementos(linha)$ 
8:    $fit \leftarrow fit + contar\_valores\_duplicados(row\_count)$ 
9: end for
10: for cada coluna em cols do
11:    $col\_count[coluna] \leftarrow contar\_elementos(coluna)$ 
12:    $fit \leftarrow fit + contar\_valores\_duplicados(col\_count)$ 
13: end for
14: retornar fit

```

Podemos observar que é apenas feita a contagem de elementos repetidos nas linhas e colunas, é possível avaliar dessa maneira porque sempre atendemos ao critério de não adicionar números repetidos numa mesma subgrade. Portanto, se em uma linha há apenas um par de números 1 repetidos, isso adicionará uma unidade ao custo total porque há uma repetição nessa linha, agora se estiver 3 números 1 repetidos na linha, será adicionado 2 ao custo total.

Figura 6. Tabuleiro de solução gulosa com os custos associados

Fitness: 17

	9	1	6	2	1	4	3	5	8	1
	3	2	4	5	8	6	1	9	7	0
	8	7	5	9	3	7	2	6	4	1
	7	8	1	4	6	2	9	4	3	1
	6	3	2	7	5	9	1	8	2	1
	4	5	9	1	3	8	6	7	5	1
	1	6	8	3	4	1	5	2	9	1
	2	4	3	9	7	5	8	3	1	1
	5	7	9	6	2	8	4	6	7	2
	0	1	1	1	1	1	1	1	1	17

Na figura 6, podemos observar em vermelho as células que, de em linha ou em coluna, adicionaram algum custo na solução final, ao final de cada linha podemos observar o custo da linha e o mesmo pode ser observado ao final de cada coluna.

3.3. Vizinhança

O operador de vizinhança utilizado nesse trabalho é simples, dada uma solução atual, escolhe-se uma subgrade e realiza a troca dos números entre duas células não fixas, dessa forma, cada subgrade continuará sem repetição de números, portanto, a restrição de repetição dentro de uma subgrade não será restringida.

Figura 7. Exemplo de uma solução vizinha



Na figura 7, podemos observar que a solução atual, que tem 41 de fitness, ao passar pelo operador de vizinhança, a subgrade esquerda do meio teve uma troca feita dos números 2 e 3 e consequentemente tendo o valor fitness sendo 38, ou seja, essa operação de troca foi vantajosa nessa ocasião.

3.3. Simulated Annealing

Simulated Annealing é um algoritmo de otimização estocástica utilizado para resolver problemas complexos. Inspirado no processo físico do recozimento de materiais, esse método busca encontrar soluções boa qualidade em espaços de busca extensos, permitindo a exploração de diferentes regiões e evitando mínimos locais de maneira que inicialmente seja feita uma abordagem quase que aleatória para tentar fazer a diversificação da solução e, conforme a temperatura for abaixando, irá realizar a intensificação a solução.

Figura 8. Pseudocódigo do Simulated Annealing utilizado

Algorithm 3: Sudoku Simulated Annealing

```

1 melhor_estado ← solução atual;
2 melhor_fitness ← melhor_estado.fit();
3 estado_atual ← solução atual;
4 iteracoes sem melhora ← 0
5 while temperatura > freezing_temperature do
6   for i in range(SA_MAX) do
7     if iter_sem_melhora > SA_MAX/2 then
8       iter_sem_melhora ← 0;
9       temperatura ← temperatura · taxa_resfriamento;
10      estado_atual ← melhor_estado;
11    end
12    iter_sem_melhora ← iter_sem_melhora + 1;
13    vizinhança ← estado_atual.obter_vizinhos;
14    melhor_vizinho ← {vizinho com menor custo};
15    fitness_atual ← estado_atual.fit();
16    fitness_vn ← vn.fit();
17    if fitness_vn < fitness_atual then
18      estado_atual ← melhor_vizinho;
19      fitness_atual ← fitness_vn;
20      if fitness_atual < melhor_fitness then
21        melhor_estado ← estado_atual;
22        melhor ← fitness_atual;
23        iter_sem_melhora ← 0
24      end
25    end
26    else if random [0,1) < exp((fitness_atual - aptidão_vn) /
27      temperatura) and fitness_atual ≠ fitness_vn then
28      estado_atual ← melhor_vizinho;
29      fitness_atual ← fitness_vn;
30    end
31    if fitness_atual = 0 then
32      break;
33    end
34  end
35  if melhor_fitness = 0 then
36    break;
37  end
38  temperatura ← temperatura · taxa_resfriamento;
39 end
return melhor_estado;

```

Tabela 2. Parâmetros utilizados no Simulated Annealing

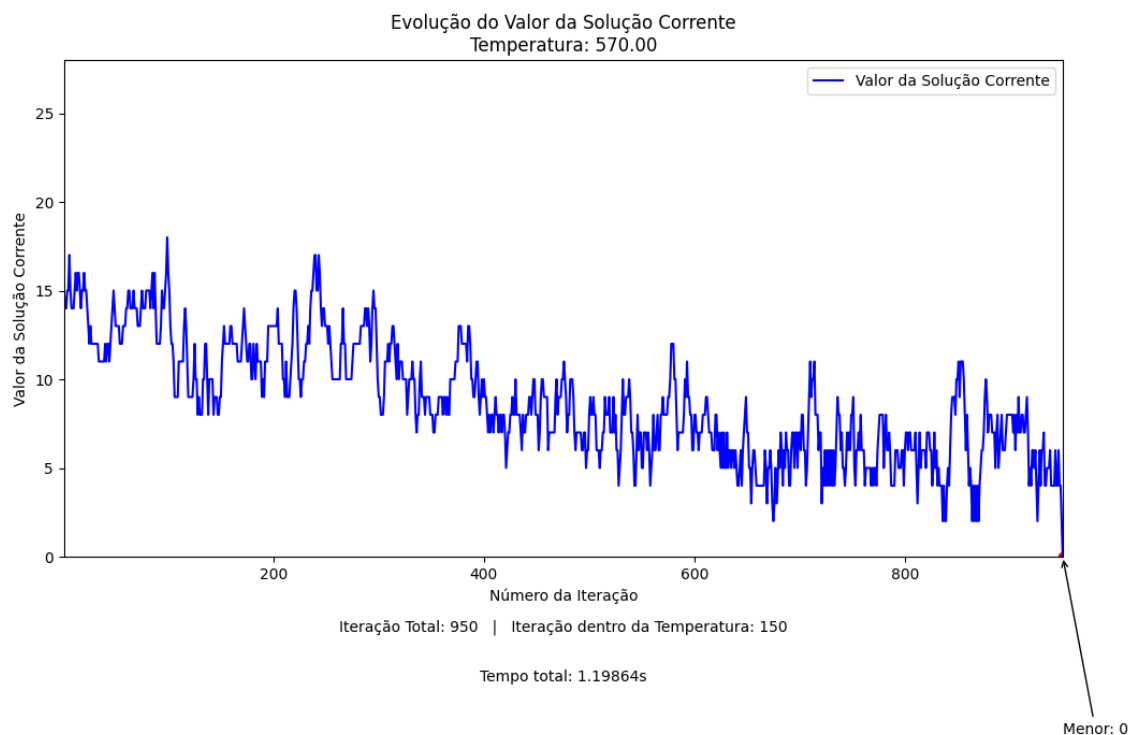
SA_MAX	Temperatura inicial	Resfriamento	Taxa de congelamento	Iterações sem melhora
800	600	0.95	0.5	400

SA_MAX: Quantidade máxima de iterações executadas em cada temperatura

Iterações sem melhora: Quantidade de iterações sem melhora necessária para que seja abaixada a temperatura forçadamente

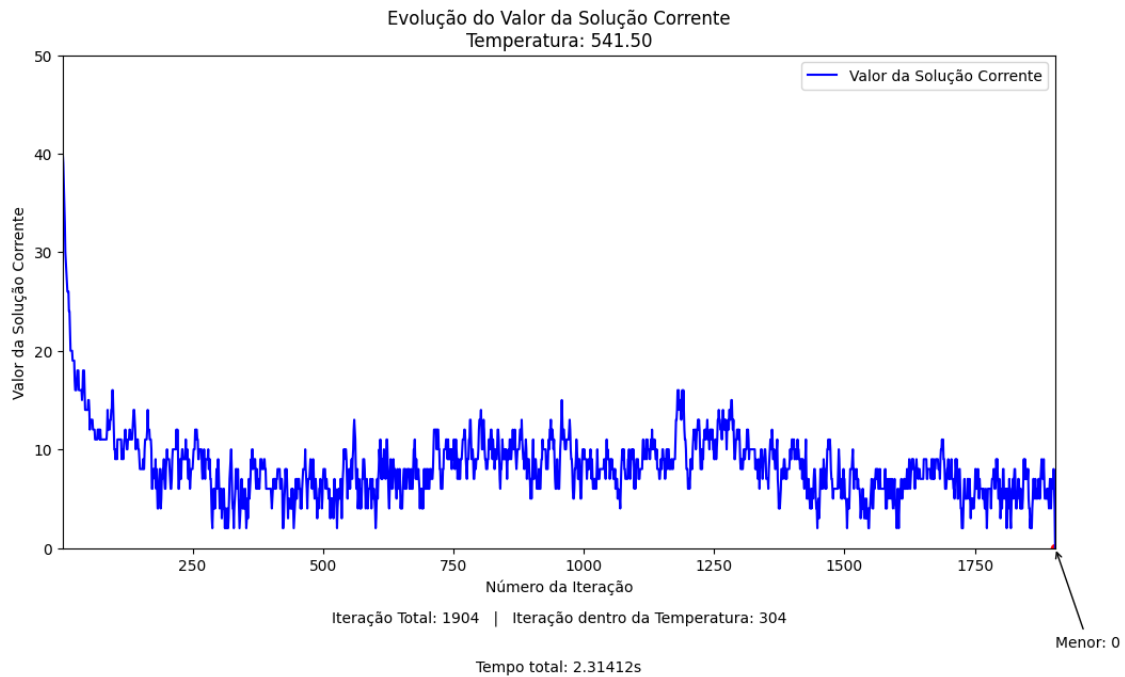
Foram os parâmetros que obtiveram os melhores resultados dentre os outros valores testados.

Figura 9. Gráfico de andamento do valor de uma solução



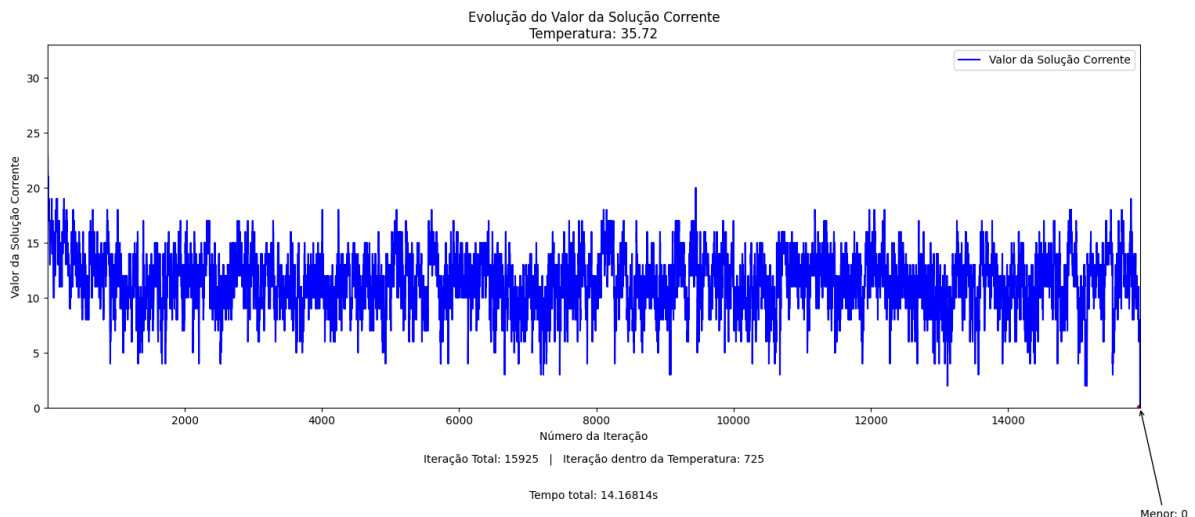
Na figura 9, podemos observar um gráfico que demonstra o andamento da resolução de uma solução inicial obtida de maneira gulosa, já na figura 10 podemos analisar o andamento da mesma instância da figura 9, entretanto com solução inicial obtida de maneira aleatória.

Figura 10. Gráfico de andamento do valor de uma solução



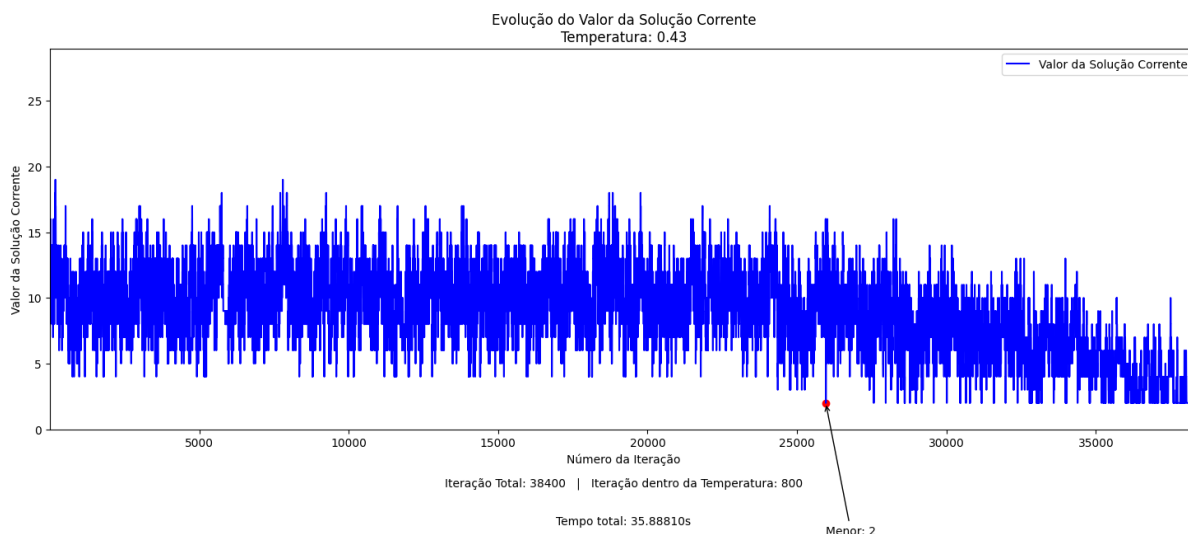
Entretanto, houveram instâncias que independente de como era obtida a solução inicial, a resolução se de maneira bastante complicada para o algoritmo, como podemos observar na figura 11.

Figura 11. Gráfico de andamento do valor de uma solução com complicações



Além disso, apenas com o Simulated Annealing não foi possível obter solução com custo zero para todas as instâncias testadas, um exemplo disso podemos observar na figura 12.

Figura 12. Gráfico de andamento do valor de uma solução não ótima ao final do algoritmo



Tendo isso em vista, iremos observar agora uma maneira que encontramos para melhorar alguns desses resultados em relação a obtenção de custo zero principalmente dessas instâncias que não foram resolvidas.

3.4. Algoritmo Genético com Simulated Annealing

Os Algoritmos Genéticos são uma técnica avançada de otimização inspirada nos princípios da seleção natural e da genética. Eles têm sido amplamente aplicados na computação evolucionária para solucionar problemas complexos de busca e otimização. Utilizando uma população de indivíduos que evoluem ao longo do tempo, por meio de operadores genéticos como cruzamento e mutação, esses algoritmos exploram e combinam diferentes soluções candidatas. Isso permite a busca por ótimos locais ou até mesmo globais.

Para este trabalho, como o Simulated Annealing não foi capaz de obter a solução ótima para todas as instâncias, a ideia é utilizar o SA como uma busca a ser feita em alguns pontos do nosso algoritmo genético, para que já sejam fornecidas soluções refinadas para o método, além disso iremos tentar resolver apenas com o decorrer das gerações

Tabela 3. Parâmetros utilizados no Algoritmo Genético

Tamanho da população	Qtde máxima de gerações	Taxa de mutação	Selecionados	Gerações sem melhora	Restart
1000	1200	0.33	500	50	150

Gerações sem melhora: Ponto em que ocorrerá a intervenção do Simulated Annealing

Restart: Quantidade de gerações em que a população é resetada

Parâmetros que obtiveram os melhores parâmetros dentre os demais testados. Podemos observar que com 0.33 de taxa de mutação, obteremos praticamente 1/3 da população tendo sofrido mutação.

Para obtenção da população inicial, foi utilizado o mesmo operador de geração de solução inicial aleatória mostrada no tópico 3.1.1, além de também ser adicionada

uma solução inicial obtida de maneira gulosa da mesma maneira em que foi apresentada no tópico 3.1.2.

Figura 13. Pseudocódigo do Algoritmo Genético + Simulated Annealing

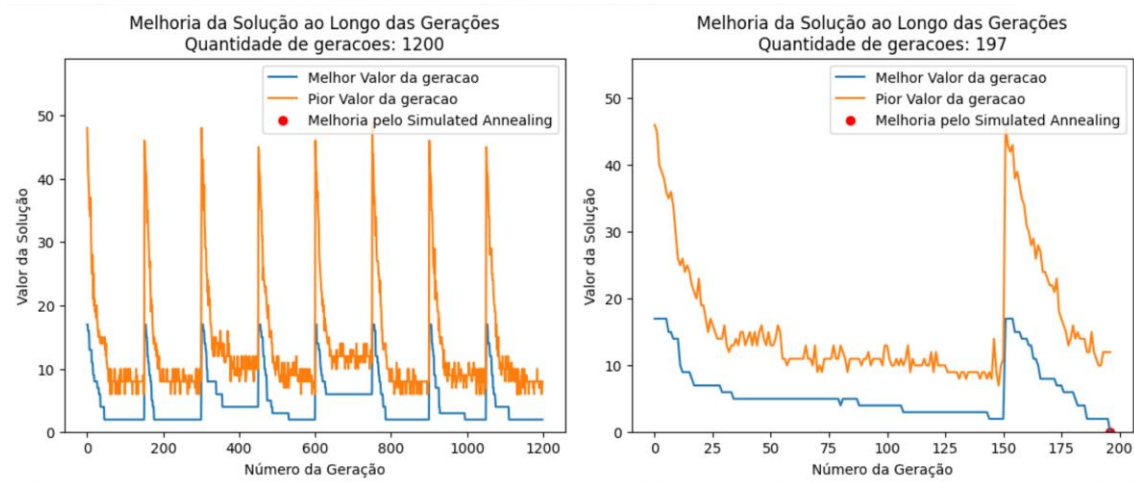
Algorithm 1 Algoritmo Genético + Simulated Annealing

```

1: procedure SOLVE
2:   gerar_população_inicial()
3:   melhor_ajuste  $\leftarrow$  população[0].fit()
4:   geração_sem_melhoria  $\leftarrow$  0
5:   geração_atual  $\leftarrow$  0
6:   for  $i$  de 0 até MAX_GERAÇÕES do
7:     selecionados  $\leftarrow$  seleção()
8:     nova_geração  $\leftarrow$  gerar_próxima_geração(selecionados)
9:     população  $\leftarrow$  copiar(nova_geração)
10:    ordenar_população()
11:    if população[0].fit() < melhor_ajuste then
12:      geração_sem_melhoria  $\leftarrow$  0
13:      melhor_ajuste  $\leftarrow$  população[0].fit()
14:    else
15:      geração_sem_melhoria  $\leftarrow$  geração_sem_melhoria + 1
16:    end if
17:    if  $i \% \text{REINICIAR\_POPULAÇÃO} = 0$  e  $i \neq 0$  then
18:      gerar_população_inicial()
19:    end if
20:    if geração_sem_melhoria > GERAÇÕES_SEM_MELHORIA then
21:      população[0]  $\leftarrow$  Simulated_Annealing(população[0])
22:      geração_sem_melhoria  $\leftarrow$  0
23:    end if
24:    if população[0].fit() = 0 then
25:      retornar população[0], geração_atual
26:    end if
27:    geração_atual  $\leftarrow$  geração_atual + 1
28:  end for
29:  retornar população[0], geração_atual
30: end procedure

```

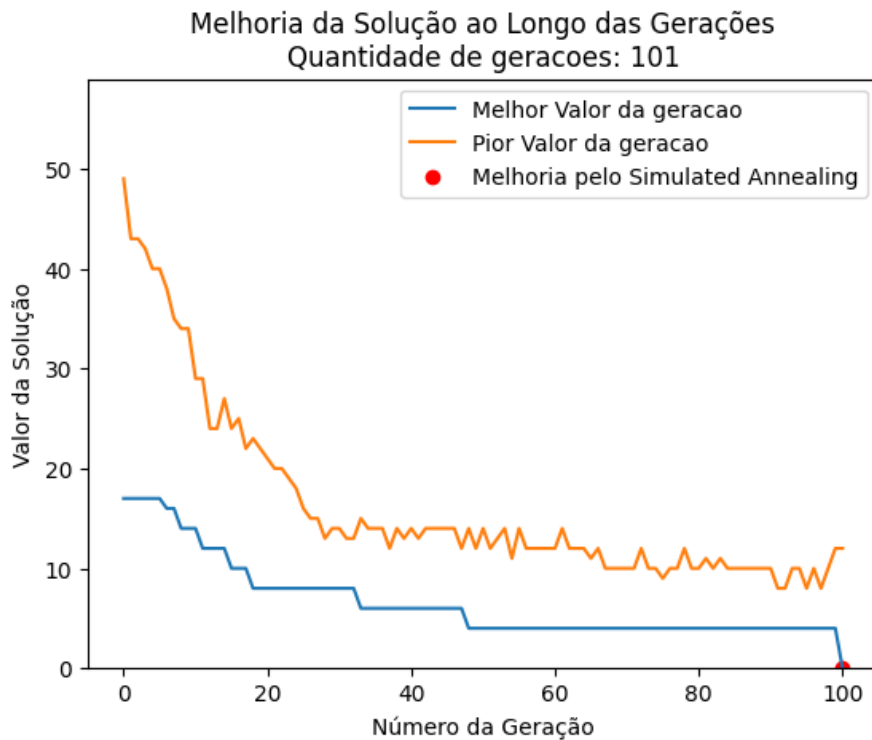
Figura 14. Comparação entre AG e AG + Busca Local



*No gráfico da direita, a legenda está "Melhoria pelo Simulated Annealing" mas, na verdade, é apenas um Hill Climbing

Na figura 14, podemos observar uma comparação entre apenas a implementação do Algoritmo Genético e o AG com uma busca local a cada 50 gerações sem melhora. Nessa instância, podemos observar que a adição de apenas uma busca local do tipo Hill Climbing conseguiu obter a solução ótima na primeira vez em que ocorre a quantidade de iterações sem melhora, entretanto a implementação não reseta essa contagem ao reiniciar a população, por isso que a visualização dessas 50 gerações é difícil visualmente.

Figura 15. AG + SA



Já na figura 15, que acontece a implementação não apenas do Hill Climbing, mas da sua forma estocástica, ou seja, o Simulated Annealing, e com isso podemos ver que há uma melhora significativa em relação a sua versão utilizada anteriormente. Nessa versão, podemos observar uma redução de praticamente metade da quantidade de gerações.

Utilizando essa abordagem do Algoritmo Genético mais o Simulated Annealing em algumas gerações, foi possível obter o resultado ótimo de todas as instâncias testadas!

Como no Algoritmo Genético há operações como crossover, mutação e seleção, precisamos entender como foram implementadas essas operações genéticas nesse trabalho.

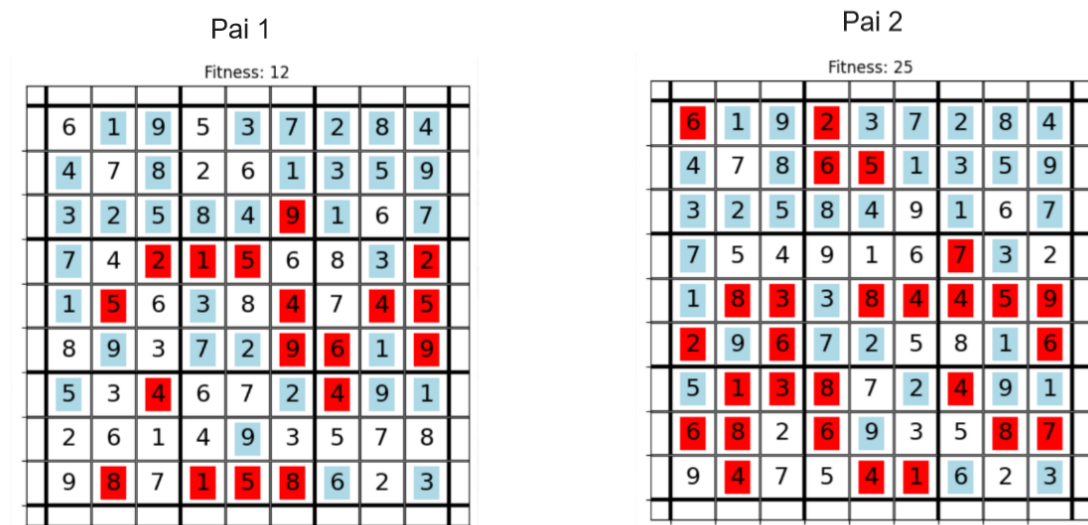
3.4.1. Crossover

Essa operação foi implementada da seguinte maneira, é obtido um número aleatório entre 1 e 8, a partir do número escolhido, os filhos são da maneira com que o primeiro tem da subgrade 0 até a subgrade de número aleatório obtido do pail e do

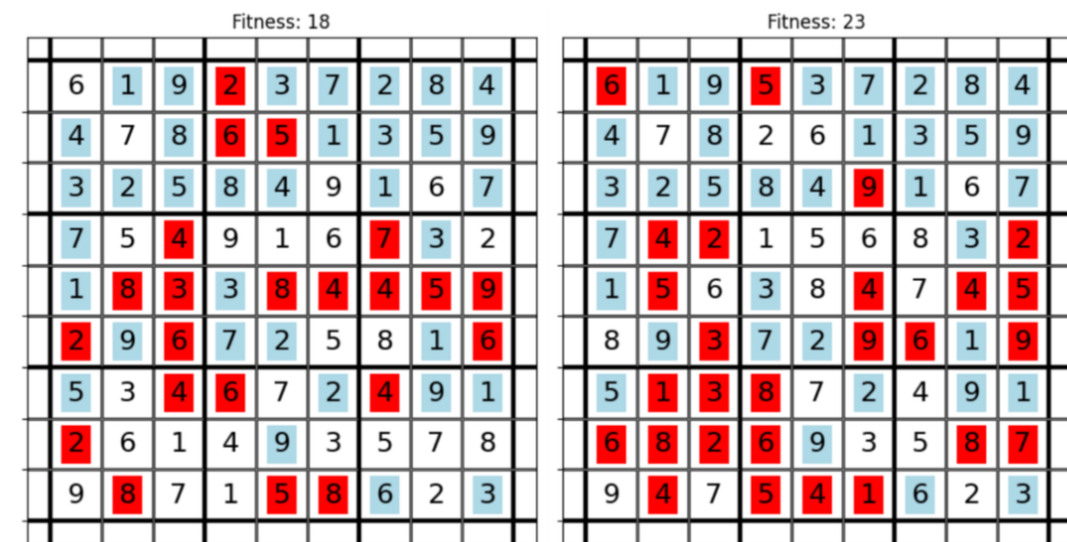
número aleatório obtido + 1 até o 8 do pai2 e o segundo filho obtém o contrário do primeiro filho.

Portanto, podemos observar que duas soluções geram duas outras soluções, por isso que a quantidade de indivíduos selecionados é metade do tamanho da população, para que dessa maneira sempre tenha uma quantidade constante de indivíduos na população, que, nesse caso, é de 1000 configurações de tabuleiro.

Figura 16. Exemplo de crossover



Grid de índice 1 foi escolhido para o crossover



3.4.2. Mutação

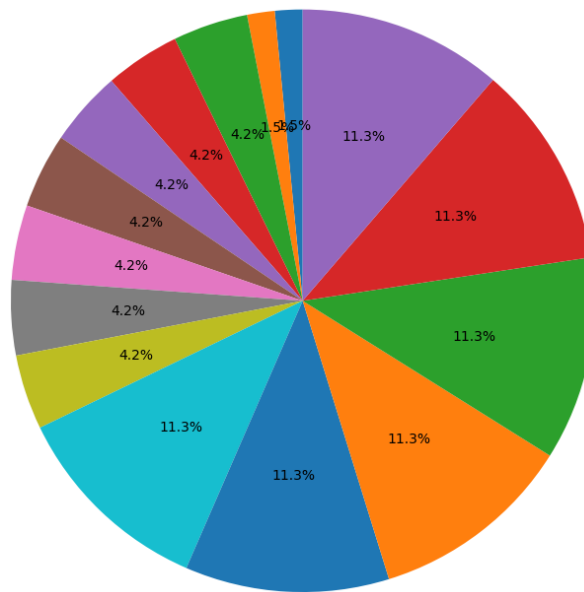
Na operação de mutação é escolhido um grid aleatório, após isso são escolhidas duas células não fixas aleatoriamente e elas têm seus números trocados, semelhante ao que ocorre no operador de vizinhança utilizado no Simulated Annealing.

Portanto um exemplo de mutação pode ser observado na figura 7.

3.4.3. Seleção

Nesse método, é obtida uma certa quantidade de indivíduos da população total baseado numa probabilidade p , p é definido baseado no custo total do indivíduo, ou seja, quanto menor o fitness, maior a chance desse indivíduo ser escolhido, essa maneira de seleção é conhecida como Roulette Wheel.

Figura 17. Exemplo de roulette wheel



Na figura 15, podemos observar uma roulette wheel com as probabilidades de indivíduo da primeira geração serem selecionados, esse gráfico foi obtido com os dados da mesma instância da figura 7.

4. Instâncias

As instâncias utilizadas para analisar a eficiência das heurísticas foram criadas utilizando a biblioteca *py-sudoku* em que é possível determinar a porcentagem de dificuldade da instância que será criada, com base na porcentagem escolhida, são removidas alguns números de células aleatórias e com isso foram classificadas instâncias em fácil (43%), médio (37%) e difícil (20%), nesse caso, para instâncias classificadas como fácil, 43% do tabuleiro está inicialmente preenchido e o mesmo se segue para as outros dois níveis.

Entretanto, com o passar dos estudos, foi observado que o nível de dificuldade de uma instância se dá não só pela quantidade de células preenchidas inicialmente, mas também por onde essas células preenchidas inicialmente estão, o que não é levado em consideração na criação de tabuleiros iniciais pela biblioteca, portanto, há instâncias caracterizadas em níveis errados.

Figura 18. Instância caracterizada como fácil erroneamente

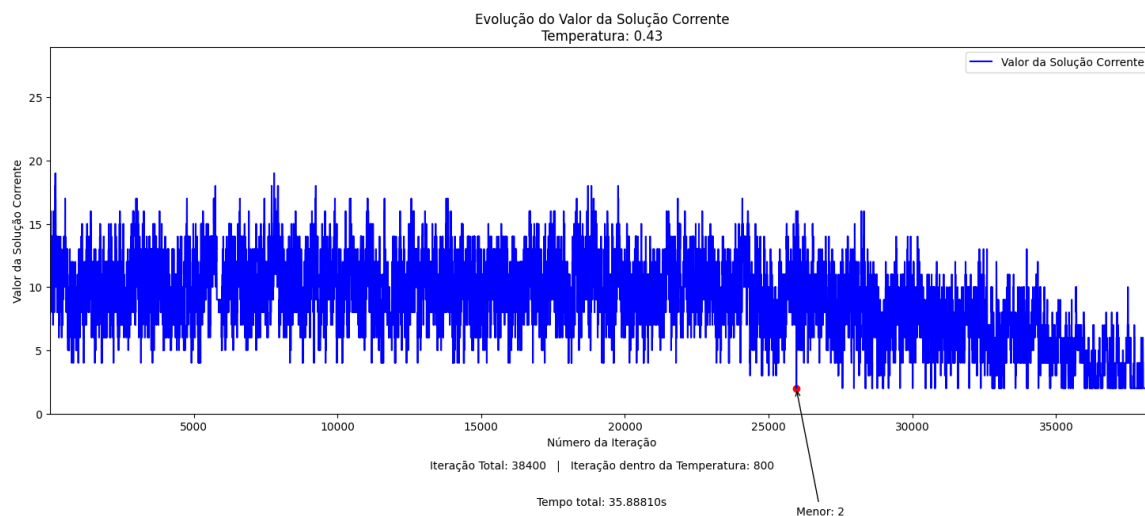
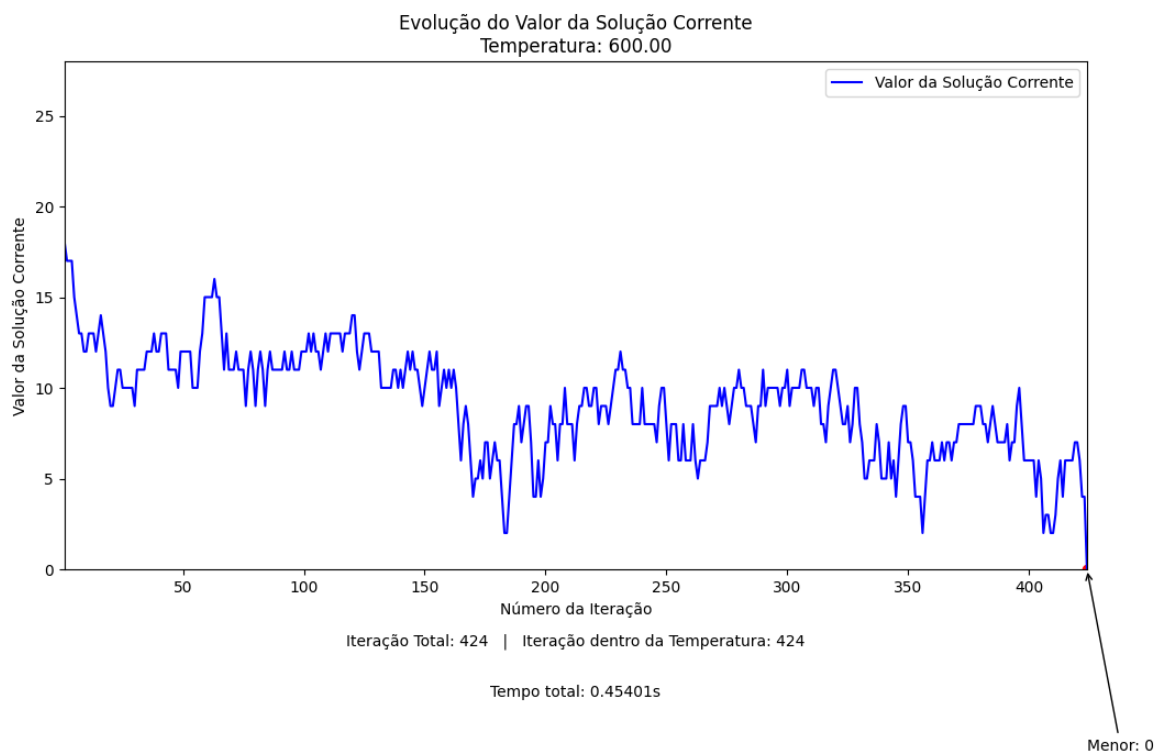


Figura 19. Instância caracterizada como difícil erroneamente



5. Resultados

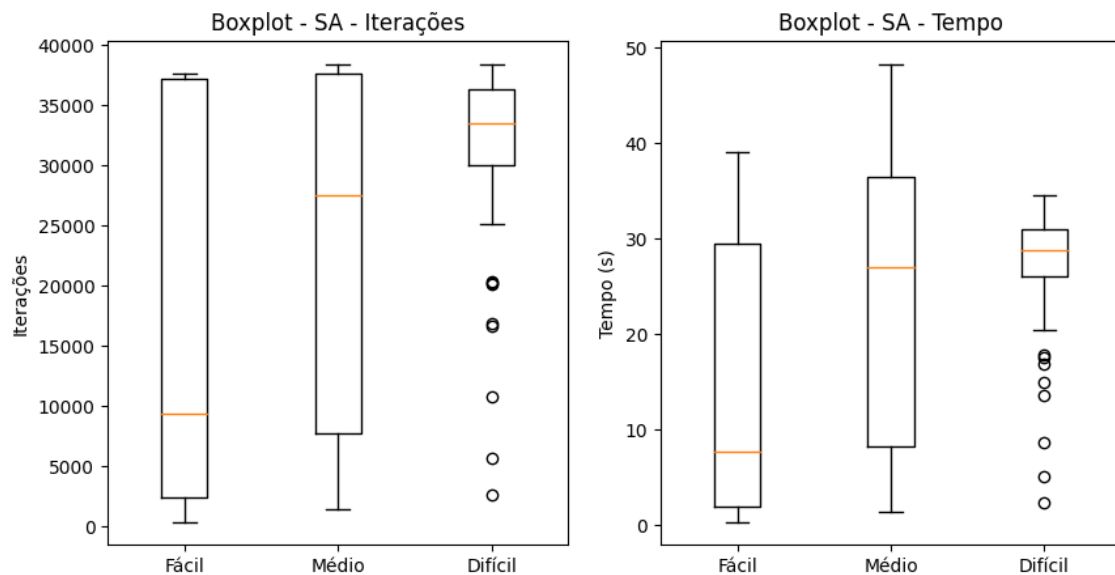
Como mencionado no tópico 4, algumas instâncias foram classificadas erroneamente, portanto algumas instâncias que foram classificadas em tal dificuldade na verdade são de outra, porém não foi feita essa divisão e esses são os resultados obtidos com essa classificação primária.

Entretanto, podemos observar que as instâncias classificadas como fáceis possuem uma diferença considerável quando comparada com as classificadas como difíceis, logo, temos fortes indícios de que a quantidade de células fixas são um fator bastante relevante para rapidez da resolução do problema.

Tabela 4. Resultados Simulated Annealing

SA	Fácil	Médio	Difícil
Média de iterações	15307.64	22461.7	28047.96
Média de tempo	14.41s	21.36s	26.11s
Maior iteração	37600	38400	38400
Menor iteração	323	197	2520
Maior tempo	41.93s	37.91s	38.87s
Menor tempo	0.26s	0.17s	2.08s
Não resolvidas	Todas resolvidas	25, 45	Todas resolvidas
Maior fitness	0	6	0

Figura 20. Boxplot Simulated Annealing

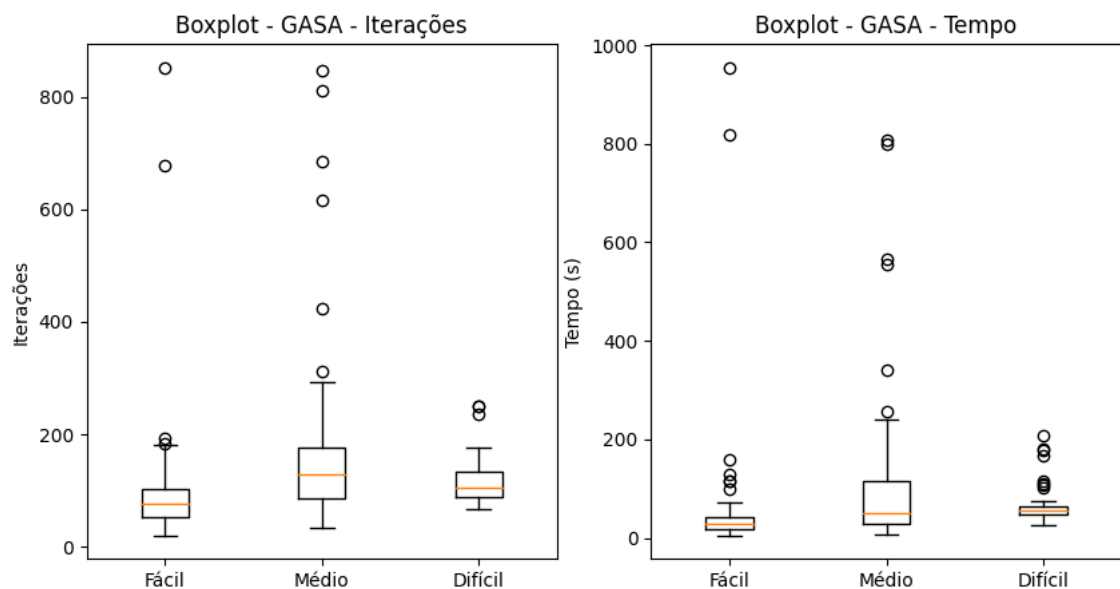


*Os resultados para a tabela e para o boxplot foram em execuções diferentes, por o Simulated Annealing possuir um fator aleatório, pode ter algumas diferenças entre os dados do boxplot e da tabela.

Tabela 5. Resultados GASA

GASA	Fácil	Médio	Difícil
Média de gerações	62.24	111.38	70.06
Média de tempo	24.60s	91.98s	40.21s
Maior geração	364	296	212
Menor geração	18	27	31
Maior tempo	476.96s	1609.06s	269.44s
Menor tempo	0.73s	2.20s	4.59s
Não resolvidas	Todas resolvidas	Todas resolvidas	Todas resolvidas
Maior fitness	0	0	0

Figura 21. Boxplot GASA



*Os resultados para a tabela e para o boxplot foram em execuções diferentes, por o Simulated Annealing possuir um fator aleatório, pode ter algumas diferenças entre os dados do boxplot e da tabela.

Com esses gráficos, podemos observar que realmente há uma classificação errada, uma vez que algumas instâncias classificadas como fáceis possuíram os maiores outliers praticamente nas execuções do GASA, mas, pelo menos, para todas instâncias testadas, classificadas erradas ou não, foi obtida sua solução ótima, ou seja, o quebra-cabeça foi resolvido!

6. Conclusão

Foi um trabalho muito interessante, uma vez que tive a oportunidade de implementar algo para resolver em menos de 1 minuto praticamente o que eu levaria mais de 10 pra conseguir terminar.

Como houve esse erro na classificação das instâncias, comparar com resultados de outros autores seria ruim também, uma vez que, em cada classificação, haveria outliers que prejudicam os resultados, portanto não seria interessante fazer esse tipo de comparação.

Ainda há melhorias possíveis para fazer com que fique melhor ainda, entretanto não consegui terminar a tempo, um exemplo seria melhorar a vizinhança, inclusive comecei essa melhora, mas não consegui consertar alguns bugs que apareceram no meio do caminho, o mesmo para o Algoritmo Genético, queria fazer com que ele, por si só resolvesse mais instâncias, porque nas que ele conseguia resolver, demorava bastantes gerações.

Um ponto que podemos observar ao longo desse trabalho é que a combinação do SA com o AG foi bastante benéfica para resolução de um quebra-cabeça.

7. Referências

Hamza, K. (2015). *A study on metaheuristic algorithms for solving Sudoku puzzles* (Master's thesis, Fen Bilimleri Enstitüsü).

DE SOUZA, Simone SF; ROMERO, Ruben. Metaheurísticas simulated annealing e busca tabu aplicadas na resolução do quebra-cabeça sudoku. **Proceeding Series of the Brazilian Society of Computational and Applied Mathematics**, v. 1, n. 1, 2013.

LEWIS, Rhyd. Metaheuristics can solve sudoku puzzles. **Journal of heuristics**, v. 13, n. 4, p. 387-401, 2007.

SIEU, Jeff. py-sudoku. Disponível em: <https://github.com/jeffsieu/py-sudoku>. Acesso em: 10 de junho de 2023

HUGHES, Ryan; YAMPOLSKIY, Roman V. Solving Sudoku Puzzles with Wisdom of Artificial Crowds. **Int. J. Intell. Games Simul.**, v. 7, n. 1, p. 24-29, 2012.

SOTO, Ricardo et al. A prefiltered cuckoo search algorithm with geometric operators for solving Sudoku problems. **The Scientific World Journal**, v. 2014, 2014.

PATHAK, Neeraj; KUMAR, Rajeev. Entropy guided evolutionary search for solving Sudoku. **Progress in Artificial Intelligence**, v. 12, n. 1, p. 61-76, 2023.

PEREZ, Meir; MARWALA, Tshilidzi. Stochastic optimization approaches for solving Sudoku. **arXiv preprint arXiv:0805.0697**, 2008.

BERTSIMAS, Dimitris; TSITSIKLIS, John. Simulated annealing. **Statistical science**, v. 8, n. 1, p. 10-15, 1993.

MANTERE, Timo; KOLJONEN, Janne. Solving, rating and generating Sudoku puzzles with GA. In: **2007 IEEE congress on evolutionary computation**. IEEE, 2007. p. 1382-1389.